rxnet — Guía de usuario: Python

FSM y Redes de Petri sobre un runtime síncrono reactivo

2026-05-20

Contents

rxnet — Guía de usuario **5**

1. El problema que resuelve rxnet . . . 5
2. La hipótesis síncrona . . . 5
3. El tick y sus fases . . . 6
    Por qué esa separación de fases . . . 6
4. Máquinas de estado finito (FSM) . . . 7
    4.1 Cuándo usar una FSM . . . 7
    4.2 Anatomía de una máquina . . . 7
    4.3 Guards y acciones . . . 8
    4.4 Ejemplo completo: luz con auto-apagado . . . 9
    4.5 El patrón de señales de entrada en FSM . . . 10
5. Redes de Petri (PN) . . . 10
    5.1 Cuándo usar una red de Petri . . . 10
    5.2 Conceptos fundamentales . . . 11
    5.3 Semántica greedy-sequential . . . 11
    5.4 Anatomía de una red . . . 12
    5.5 Lugares de señal (signal places) . . . 12
    5.6 Ejemplo completo: blink con tres velocidades . . . 13
6. Modelos de ejecución concurrente . . . 14
    6.1 Ejecutivo cíclico — `CyclicExecutive` . . . 14
    6.2 Multitarea cooperativa — `CoopExecutive` . . . 15
    6.3 Threads paralelos — `ThreadExecutive` . . . 15
    6.4 Resumen comparativo de executors . . . 16
    6.5 Multi-rate: nodos a distintas frecuencias . . . 17
    6.6 Comunicación entre nodos . . . 17
    6.7 Acciones diferidas asíncronas (WorkerPool) . . . 18
7. Cuándo usar FSM, cuándo PN . . . 18
    Usa FSM cuando… . . . 18
    Usa PN cuando… . . . 19
    Mezclar ambos (patrón habitual) . . . 19

8. Referencia rápida de la API 20
Core runtime 20
FSM 21
Petri Net 21
Firmas de callbacks 22
9. Depuración y trazado 22
Cómo funciona 23
Uso básico 23
Descargar y visualizar la traza 23
Trazado de fases (para docencia) 24
Eventos de usuario 24
Nombres de estados y lugares 25
Retirar el trazado 25
10. Extracción de diagramas y verificación formal 25
Instalación 25
Uso desde la línea de comandos 26
Cómo funciona 26
Ejemplo de salida Mermaid 26
Restricción: solo patrones estáticos 27
Lectura adicional 27

List of Figures

rxnet — Guía de usuario

1. El problema que resuelve rxnet

Los sistemas reactivos responden a eventos del entorno: pulsaciones de botón, lecturas de sensor, mensajes de red, expiración de temporizadores. El reto no es *qué hacer* ante un evento, sino *cuándo hacerlo* y *qué hay que ver* en ese momento.

Sin disciplina, el código reactivo acumula dos tipos de bugs difíciles de reproducir:

- **Race conditions de lectura**: el guard de una transición lee un bit de hardware a mitad de ciclo, cuando otro módulo ya lo consumió.
- **Efectos secundarios prematuros**: una acción ejecutada durante la evaluación modifica estado que otro módulo todavía no evaluó.

rxnet impone una disciplina que los elimina por construcción: **toda la información que entra en el sistema se lee exactamente una vez por ciclo, al principio, y todos los efectos secundarios se ejecutan exactamente una vez, al final**.

---

2. La hipótesis síncrona

rxnet está inspirada en los *lenguajes síncronos* (Esterel, Lustre, Signal) y en sus derivados industriales (SCADE, Simulink). La idea central se llama la **hipótesis síncrona**:

> La computación de un tick es instantánea. El mundo externo no cambia mientras el sistema está procesando.

Esto es una abstracción, no una afirmación física. En la práctica significa:

- El tick debe completarse antes de que llegue el siguiente evento significativo.
- Si el tick tarda más que el período de muestreo, el sistema tiene un problema de diseño, no de concurrencia.

Bajo la hipótesis síncrona, **no hay carrera de datos posible dentro del tick**: todos los módulos leen el mismo snapshot de entradas, y nadie ve los cambios de los demás hasta el siguiente tick.

3. El tick y sus fases

Cada llamada a `runtime.tick()` ejecuta cuatro fases de nodo en orden estricto:

#	Fase	Qué hace
1	**Latch**	`latch_inputs_cb(ctx, user)` — leer GPIO, calcular señales derivadas, tomar snapshot
2	**Evaluate**	`evaluate()` — decidir siguiente estado / flags (solo lectura del snapshot)
3	**Commit**	`commit()` — aplicar decisiones, encolar acciones diferidas
4	**Dump**	`dump_outputs_cb(ctx, user)` — escribir GPIO, imprimir estado

Entre `commit` y `dump`, el runtime despacha las acciones diferidas encoladas. Ese despacho es una barrera de semántica, no una fase de nodo independiente.

Sub-paso exclusivo de Python: antes de llamar a los callbacks por nodo, el runtime ejecuta `context.latch_inputs()`, que copia `ctx.inputs` (dict de entradas en vivo, modificable desde cualquier hilo) a `ctx.latched_inputs` (snapshot de solo lectura para este tick). Esto permite compartir señales globales entre nodos sin riesgo de carrera.

Por qué esa separación de fases

La separación **evaluate / commit / dump** no es arbitraria:

- **Evaluate** solo puede leer, nunca escribir estado observable. Todos los módulos ven el mismo estado del sistema al mismo tiempo.

- **Commit** aplica las decisiones. Tras el commit de todos los nodos, el runtime ejecuta las acciones diferidas antes de cualquier dump. Una acción ve el estado comprometido de todos los módulos, no solo el propio.
- **Dump** publica salidas después de commit y del despacho de acciones diferidas.

La separación **latch / dump** garantiza que las lecturas de hardware ocurren *antes* de evaluar (snapshot limpio) y las escrituras *después* de decidir (salida consistente).

---

4. Máquinas de estado finito (FSM)

4.1 Cuándo usar una FSM

Una FSM es la herramienta adecuada cuando el comportamiento del módulo depende de su **historia**: no solo del evento actual sino de lo que pasó antes.

Ejemplos típicos:

Problema	Estados naturales
Luz con botón	OFF, ON
Semáforo	ROJO, VERDE, AMARILLO
Puerta con cerrojo	ABIERTA, CERRADA, BLOQUEADA
Protocolo de comunicación	IDLE, CONECTANDO, CONECTADO, ERROR
Menú de display	MENU_PRINCIPAL, SUBMENU_A, SUBMENU_B

4.2 Anatomía de una máquina

```
from rxnet.fsm import Machine, Transition, Runtime

# Estados: constantes enteras (el runtime no sabe qué significan)
IDLE    = 0
RUNNING = 1
ERROR   = 2

# Transiciones: (estado_origen, estado_destino, guardia?, acción?)
transitions = [
```

```
    Transition(IDLE,    RUNNING, guard=button_pressed, action=
        start_motor),
    Transition(RUNNING, IDLE,    guard=button_pressed, action=
        stop_motor),
    Transition(RUNNING, ERROR,   guard=fault_detected, action=
        report_fault),
    Transition(ERROR,   IDLE,    guard=reset_pressed,  action=
        clear_fault),
]

machine = Machine(
    name="motor",
    state=IDLE,
    transitions=transitions,
    user=my_data,              # datos de usuario, pasados a guard/
        action
    latch_inputs_cb=read_gpio, # fase latch: snapshot hardware
    dump_outputs_cb=write_gpio,# fase dump: escribir hardware
)

rt = Runtime()
rt.add_machine(machine)
```

Semántica first-match: en cada tick, el runtime recorre las transiciones en orden de declaración y ejecuta la *primera* cuya guardia sea verdadera y cuyo estado origen coincida con el estado actual. Si ninguna coincide, la máquina permanece en su estado.

4.3 Guards y acciones

```
from rxnet.runtime import Context

# Guard: función pura que devuelve bool
# Recibe el contexto del tick y los datos de usuario
def button_pressed(ctx: Context, data: MyData) -> bool:
    return data.latched_button_event  # leer snapshot, nunca hardware
        en vivo

# Acción: función deferred (corre tras commit, antes de dump)
# En este punto todos los módulos ya aplicaron sus cambios
def start_motor(ctx: Context, data: MyData) -> None:
    data.motor_enabled = True
    # Aquí podemos consultar otros módulos: su estado ya está
        comprometido
```

Regla importante: los guards nunca deben tener efectos secundarios. Son funciones puras de lectura. Los efectos van en las acciones.

4.4 Ejemplo completo: luz con auto-apagado

```python
import time
from rxnet.fsm import Machine, Transition, Runtime
from rxnet.runtime import Context

# -- Estados --
OFF = 0
ON  = 1

# -- Datos de la máquina --
class LightData:
    def __init__(self, timeout_ms: int):
        self.latched_event = False
        self.enabled = False
        self.timeout_ms = timeout_ms
        self.deadline_ms = 0
        self.now_ms = 0

def now_ms() -> int:
    return int(time.monotonic() * 1000)

# -- Callbacks de fase --
def latch(ctx: Context, d: LightData) -> None:
    d.now_ms = now_ms()
    d.latched_event = read_button()   # snapshot del hardware

def dump(ctx: Context, d: LightData) -> None:
    set_led(d.enabled)

# -- Guards --
def pressed(ctx: Context, d: LightData) -> bool:
    return d.latched_event

def timed_out(ctx: Context, d: LightData) -> bool:
    return d.now_ms >= d.deadline_ms

# -- Acciones (deferred: ven el estado comprometido) --
def turn_on(ctx: Context, d: LightData) -> None:
    d.enabled = True
    d.deadline_ms = now_ms() + d.timeout_ms

def turn_off(ctx: Context, d: LightData) -> None:
    d.enabled = False

# -- Montaje --
data = LightData(timeout_ms=5000)
machine = Machine(
    name="light",
    state=OFF,
    transitions=[
```

```
        Transition(OFF, ON,  guard=pressed,   action=turn_on),
        Transition(ON,  ON,  guard=pressed,   action=turn_on),  # reset
            timer
        Transition(ON,  OFF, guard=timed_out, action=turn_off),
    ],
    user=data,
    latch_inputs_cb=latch,
    dump_outputs_cb=dump,
)

rt = Runtime()
rt.add_machine(machine)

# Bucle principal (10 ms)
while True:
    rt.tick()
    time.sleep(0.010)
```

4.5 El patrón de señales de entrada en FSM

La forma canónica de conectar hardware a una FSM en rxnet:

Hardware	`latch_inputs_cb`	guards / actions
GPIO → evento	`data.latched = True`	`if data.latched: ...`
(vivo, volátil)	(snapshot estable)	(solo leen snapshot)

El callback `latch_inputs_cb` es el único punto de contacto con el hardware. El resto de la máquina trabaja con copias estables.

---

5. Redes de Petri (PN)

5.1 Cuándo usar una red de Petri

Una red de Petri es la herramienta adecuada cuando hay **concurrencia natural**: varios procesos que avanzan en paralelo y necesitan sincronizarse.

Ejemplos típicos:

Problema	Modela bien porque…
Botón toggle	Un token fluye entre P_OFF y P_ON
Buffer productor-consumidor	Tokens representan slots llenos/vacíos
Semáforo / mutex	Un token en P_LIBRE controla el acceso
Protocolo de handshake	Ambos lados avanzan y se sincronizan
Recursos compartidos	Tokens cuentan instancias disponibles

La PN no reemplaza a la FSM: cuando el comportamiento es esencialmente lineal (un agente con historia), la FSM es más clara. Cuando hay múltiples flujos que se sincronizan, la PN es más clara.

5.2 Conceptos fundamentales

Concepto	Símbolo	Significado
Lugar (place)	○	condición, estado, recurso, mensaje pendiente
Token	•	unidad de recurso, condición activa
Transición	rect.	evento, paso de proceso
Arco de entrada	→T	condición necesaria para disparar (consume tokens)
Arco de salida	T→	condición que produce al disparar (produce tokens)

Una transición **está habilitada** cuando todos sus lugares de entrada tienen suficientes tokens (≥ peso del arco). Una transición habilitada **dispara**: consume los tokens de sus entradas y produce tokens en sus salidas.

5.3 Semántica greedy-sequential

rxnet usa evaluación **greedy-sequential**: las transiciones se evalúan en orden de declaración, y las anteriores *consumen tokens inmediatamente*, antes de que las posteriores se evalúen.

Consecuencia práctica: **el orden de declaración implica prioridad**. Si dos transiciones compiten por el mismo token, gana la que se declara primero.

```
transitions = [
    # T0 declarada antes → prioridad sobre T1
    Transition(consume=[Arc(P_X, 1)], produce=[Arc(P_A, 1)]),  # T0
    Transition(consume=[Arc(P_X, 1)], produce=[Arc(P_B, 1)]),  # T1 (no
        dispara si T0 disparó)
]
```

5.4 Anatomía de una red

```
from rxnet.pn import Net, Transition, Arc, Runtime

# Lugares: índices 0, 1, 2, ...
P_OFF     = 0
P_ON      = 1
P_REQUEST = 2

net = Net(
    name="light",
    places=[1, 0, 0],   # token inicial en P_OFF
    transitions=[
        # off + request → on
        Transition(consume=[Arc(P_OFF, 1), Arc(P_REQUEST, 1)],
                   produce=[Arc(P_ON, 1)]),
        # on + request → off
        Transition(consume=[Arc(P_ON, 1), Arc(P_REQUEST, 1)],
                   produce=[Arc(P_OFF, 1)]),
    ],
)

rt = Runtime()
rt.add_net(net)
```

5.5 Lugares de señal (signal places)

Un **lugar de señal** no acumula tokens: se *restablece* en cada latch a 0 o 1 según una condición del entorno. Es el equivalente PN de los inputs latched de una FSM.

```
# En latch_inputs_cb: reescribir P_TOGGLE_DUE en cada tick
def latch_cb(ctx, _user):
    is_blinking = net.places[P_X1] > 0 or net.places[P_X2] > 0
    net.places[P_TOGGLE_DUE] = 1 if (is_blinking and timer_elapsed())
       else 0
```

Contrasta con los **lugares de cola** (como P_REQUEST), que acumulan tokens:

```
# En latch_inputs_cb: añadir un token si hay pulsación
def latch_cb(ctx, _user):
    if button_pressed():
        net.places[P_REQUEST] += 1   # acumula; se consume 1 por tick
```

Tipo de lugar	Comportamiento	Ejemplo
Señal	Se reescribe a 0/1 cada latch	Timer expirado, sensor activo
Cola	Acumula; transición consume 1	Pulsación de botón pendiente
Estado	Token único que fluye	ON/OFF, ROJO/VERDE
Contador	N tokens = N recursos	Slots libres en buffer

5.6 Ejemplo completo: blink con tres velocidades

El botón avanza cíclicamente entre modos: OFF → X1 → X2 → OFF. Dentro de X1 y X2 hay un self-loop de toggle que invierte el LED cada semiperíodo (a velocidad base en X1, al doble en X2).

```
P_OFF, P_X1, P_X2, P_REQUEST, P_TOGGLE_DUE = 0, 1, 2, 3, 4

def create_blink_pn(button_gpio, light_gpio, base_hz):
    state = {"output": False, "next_toggle_ms": 0, "now_ms": 0}

    def action_enter_x1(ctx, _): state["output"] = True; ...
    def action_enter_x2(ctx, _): state["output"] = True; ...
    def action_enter_off(ctx, _): state["output"] = False; state["
        next_toggle_ms"] = 0
    def action_toggle(ctx, _): state["output"] = not state["output"];
        ...

    net = Net(
        name="blink",
        places=[1, 0, 0, 0, 0],
        transitions=[
            # Transiciones de botón ANTES que las de toggle (prioridad
                greedy)
            Transition(consume=[Arc(P_OFF,1), Arc(P_REQUEST,1)],
                produce=[Arc(P_X1,1)], action=action_enter_x1),
            Transition(consume=[Arc(P_X1,1), Arc(P_REQUEST,1)], produce
                =[Arc(P_X2,1)], action=action_enter_x2),
            Transition(consume=[Arc(P_X2,1), Arc(P_REQUEST,1)], produce
                =[Arc(P_OFF,1)], action=action_enter_off),
            # Self-loops de toggle DESPUÉS (pierden si hay botón en el
                mismo tick)
```

```
            Transition(consume=[Arc(P_X1,1), Arc(P_TOGGLE_DUE,1)],
                produce=[Arc(P_X1,1)], action=action_toggle),
            Transition(consume=[Arc(P_X2,1), Arc(P_TOGGLE_DUE,1)],
                produce=[Arc(P_X2,1)], action=action_toggle),
        ],
    )
    # ... latch/dump callbacks
    return net
```

---

6. Modelos de ejecución concurrente

rxnet incluye tres **executors** que gestionan el timing y el scheduling automáticamente. El período base de cada runtime se calcula como el MCD de los períodos de sus nodos (`add_machine(m, period_us=...)` / `add_node(n, period_us=...)`). El runtime no materializa una tabla de activaciones; cada executor construye o mantiene la estructura que necesita. Antes de `latch`, el executor escribe el instante lógico del grupo en `ctx.activation_us`, común para todos los nodos activados en ese instante.

6.1 Ejecutivo cíclico — `CyclicExecutive`

Tabla de despacho estática con hiperperíodo. Un único hilo, orden de slots determinista. Adecuado para la mayoría de los casos.

```
from rxnet.cyclic import CyclicExecutive
from rxnet.fsm import Machine, Runtime

rt = Runtime()
rt.add_machine(machine, period_us=10_000)   # 10 ms

ce = CyclicExecutive()
ce.add(rt)
ce.run()   # retorna cuando ce.stop() es llamado
```

Para terminar, llama a `ce.stop()` desde cualquier acción, guard, o hilo externo:

```
def check_exit(ctx, data):
    if data.should_exit:
        ce.stop()
```

El executor calcula automáticamente: - `base = MCD(períodos)` → período base - `hyper = MCM(períodos)` → hiperperíodo, N slots en su propia tabla

Cuándo usarlo: períodos fijos, determinismo máximo, hilo único.

Limitaciones: si los períodos no son armónicos, el hiperperíodo puede crecer mucho; en ese caso suele convenir `CoopExecutive` o `ThreadExecutive`.

6.2 Multitarea cooperativa — `CoopExecutive`

Scheduling dinámico por deadline. Un único hilo ejecuta los nodos vencidos por orden de deadline y duerme hasta la siguiente activación. No requiere que los períodos sean múltiplos entre sí.

```
from rxnet.coop import CoopExecutive

ce = CoopExecutive()
ce.add(fast_rt)   # período 10 ms
ce.add(slow_rt)   # período 15 ms
ce.run()   # retorna cuando ce.stop() es llamado
```

El executor avanza cada activación desde su último disparo, evitando la acumulación de deriva de fase incluso cuando un nodo se retrasa.

Cuándo usarlo: períodos irregulares, tareas que a veces se alargan un poco, sin overhead de threads.

Limitaciones: una tarea muy lenta puede retrasar a todas las demás.

6.3 Threads paralelos — `ThreadExecutive`

Un thread por nodo periódico. Cada grupo de activación usa dos barreras BSP: una tras `evaluate` y otra tras `commit` más el despacho de acciones diferidas. El último nodo del último runtime corre en el hilo llamante (útil para nodo CLI/stdin). Los nodos async no están soportados en `ThreadExecutive`.

```
from rxnet.thread import ThreadExecutive

te = ThreadExecutive()
te.add(pn_rt)    # nodos PN → threads en background
te.add(cli_rt)   # CLI → hilo principal
te.run()   # retorna cuando te.stop() es llamado
```

Importante: nunca llames a `sys.exit()` directamente desde dentro de `ThreadExecutive` — los threads de background quedarían bloqueados en las barreras. Usa siempre `te.stop()` para terminar limpiamente.

Cuándo usarlo: varios nodos con trabajo de cómputo intensivo; sistemas con múltiples cores.

Limitaciones: overhead de barreras; Python GIL limita el paralelismo real en código puro (pero no en I/O ni extensiones nativas).

El análisis automático de planificabilidad se activa con `enable_sched_check(True)` y puede reportarse con `check_schedulability()`. `CyclicExecutive` analiza su hiperperíodo con WCETs medidos, `CoopExecutive` usa análisis iterativo de tiempo de respuesta con bloqueo cooperativo, y `ThreadExecutive` usa análisis de tiempo de respuesta con prioridades fijas por plazo efectivo.

Las secciones críticas sobre recursos compartidos se declaran dentro del callback del nodo con `ctx.critical_section(resource_id)`. En `ThreadExecutive` el contexto protege la sección con un mutex por recurso; en `CyclicExecutive` y `CoopExecutive` no toma ningún mutex porque la ejecución ya está contenida en el bloque atómico del tick, pero sí mide el tiempo. El runtime guarda el máximo observado para cada par tarea/recurso y lo añade al informe de planificabilidad.

```
def evaluate(self, ctx):
    with ctx.critical_section(1):
        shared_state.value += 1
```

En el informe, `B` es el bloqueo máximo, `I` la interferencia máxima y `R` el tiempo de respuesta peor caso. Para `ThreadExecutive`, `B` se calcula como la suma máxima de combinaciones de secciones críticas de tareas de menor prioridad, sin usar dos secciones de la misma tarea ni dos secciones del mismo recurso. La interferencia se calcula iterativamente: primero se asume una activación de cada tarea de mayor prioridad en el instante crítico, y después se recalcula `I = sum(ceil(R/Tj)*Cj)` con el último `R = C + B + I` hasta que el tiempo de respuesta converge o supera el plazo.

6.4 Resumen comparativo de executors

	`CyclicExecutive`	`CoopExecutive`	`ThreadExecutive`
Threads	1	1	1 por nodo periódico
Dispatch	Tabla estática (hiperperíodo del executive)	Próxima activación + deadline	Grupos de activación + barreras BSP
Períodos	Mejor con períodos armónicos o hiperperíodo corto	Cualquiera	Cualquiera
Paralelismo	No	No	Sí (I/O y extensiones)

	`CyclicExecutive`	`CoopExecutive`	`ThreadExecutive`
Overhead	Mínimo	Mínimo	Barreras mutex
Casos típicos	Mayoría de casos	Períodos irregulares	Múltiples cores

Todos los executors ofrecen `on_stop()` para ejecutar limpieza antes de que `run()` retorne:

```
ce.on_stop(lambda: pool.shutdown(wait=True))
```

6.5 Multi-rate: nodos a distintas frecuencias

Los executors gestionan el multi-rate automáticamente a partir de los períodos declarados al registrar los nodos:

```
rt = Runtime()
rt.add_machine(fast_machine, period_us=10_000)    # 10 ms
rt.add_machine(slow_machine, period_us=100_000)   # 100 ms

ce = CyclicExecutive()
ce.add(rt)
ce.run()
```

Con `CyclicExecutive`, el executor calcula: - `base = MCD(10 ms, 100 ms)= 10 ms` - `hyper = MCM(10 ms, 100 ms)= 100 ms` → 10 slots - Slot 0: `fast_machine + slow_machine`; Slots 1–9: solo `fast_machine`

Para runtimes completamente independientes a distintas frecuencias:

```
fast_rt = Runtime()
fast_rt.add_machine(control_machine, period_us=1_000)    # 1 ms

slow_rt = Runtime()
slow_rt.add_machine(monitor_machine, period_us=100_000)  # 100 ms

ce = CoopExecutive()   # o CyclicExecutive
ce.add(fast_rt)
ce.add(slow_rt)
ce.run()
```

6.6 Comunicación entre nodos

Estado de FSM como señal: un nodo puede leer `machine_a.state` directamente en sus guards — el estado fue comprometido en commit, antes del siguiente tick.

Tokens de PN como mensajes: el productor escribe `net.places[P_BUFFER]` en su `latch_inputs_cb`; el consumidor dispara la transición que consume ese token.

Acciones diferidas: cualquier callback puede encolar una acción para la despacho deferred del tick actual:

```
ctx.enqueue_deferred_action(my_action_fn, user)
ctx.enqueue_deferred_action(send_alarm, user, Priority.CRITICAL)
```

6.7 Acciones diferidas asíncronas (WorkerPool)

Para acciones de larga duración (peticiones HTTP, acceso a disco) que no deben bloquear el tick:

```
from rxnet.worker_pool import WorkerPool
from rxnet.runtime import Runtime

pool = WorkerPool(num_workers=4)
rt = Runtime(worker_pool=pool)

ce = CyclicExecutive()
ce.add(rt)
ce.on_stop(lambda: pool.shutdown(wait=True))
ce.run()
```

Con un `WorkerPool` activo, `tick()` **no bloquea** en el despacho deferred: publica las acciones al pool y pasa directamente a dump. Los resultados llegan a `ctx.inputs` en el siguiente latch.

Nivel	Valor	Uso típico
`Priority.CRITICAL`	3	Alarmas, paradas de emergencia
`Priority.HIGH`	2	Control en tiempo real
`Priority.NORMAL`	1	Lógica de aplicación (defecto)
`Priority.LOW`	0	Telemetría, logging, estadísticas

---

7. Cuándo usar FSM, cuándo PN

Usa FSM cuando...

- El módulo tiene **un agente con historia**: hay un único “estado del módulo” en cada momento.

- Las transiciones son mutuamente excluyentes: estar en un estado excluye estar en otro.
- El control de flujo es lineal (incluso si tiene bucles y ramas).
- Necesitas guards complejos que leen múltiples variables.

Ejemplos:

- Semáforo: ROJO → VERDE → AMARILLO → ROJO
- Cerrojo: DESBLOQUEADO → BLOQUEADO
- Protocolo: IDLE → SYN_SENT → ESTABLISHED

Usa PN cuando...

- Hay **múltiples agentes independientes** que se sincronizan.
- Los recursos son contables (N slots, M conexiones disponibles).
- El paralelismo es natural: varias actividades ocurren al mismo tiempo.
- Necesitas modelar producción/consumo de forma explícita.

Ejemplos:

- Productor/consumidor: places `P_LLENOS` + `P_VACIOS`
- Semáforo binario: `P_MUTEX` (0 o 1 token)
- Rendezvous: A espera a B, B espera a A

Mezclar ambos (patrón habitual)

Lo más potente es combinarlos. La FSM describe la **política** (qué modo estoy); la PN describe los **recursos y sincronización** (qué tengo disponible):

```
# Sistema de acceso a sala:
# FSM: gestiona el ciclo de apertura de puerta
# PN:  gestiona los recursos (tokens = personas dentro)

door_machine = Machine(...)
capacity_net = Net(...)

rt = Runtime()
rt.add_machine(door_machine)
rt.add_net(capacity_net)

while True:
    rt.tick()        # FSM y PN avanzan juntos en el mismo tick
    time.sleep(0.010)
```

---

8. Referencia rápida de la API

Core runtime

```python
from rxnet.runtime import Context, Runtime
from rxnet.worker_pool import Priority, WorkerPool
from concurrent.futures import ThreadPoolExecutor

# Creación del runtime (todos los parámetros son opcionales)
rt = Runtime(
    inputs=my_inputs,               # snapshot global (cualquier tipo)
    executor=ThreadPoolExecutor(4), # tick paralelo por fases
    worker_pool=WorkerPool(4),      # acciones deferred asíncronas
)

rt.add_node(node, period_us=10_000, deadline_us=10_000)
rt.tick()                           # tick manual: ejecuta todos los nodos
rt.tick_nodes_at([0, 1], activation_us=20_000)  # uso interno de executors
rt.context                          # acceder al contexto

# Contexto
ctx.inputs                          # entradas en vivo (mutables desde fuera)
ctx.latched_inputs                  # snapshot de este tick (solo lectura)
ctx.activation_us                   # instante lógico del grupo activo

# Encolar acción deferred — prioridad por defecto NORMAL
ctx.enqueue_deferred_action(fn, user)
ctx.enqueue_deferred_action(fn, user, Priority.HIGH)   # con prioridad explícita

# WorkerPool
from rxnet.worker_pool import Priority, WorkerPool

with WorkerPool(num_workers=4) as pool:
    pool.submit(fn, ctx, user, Priority.NORMAL)  # enviar manualmente
    pool.shutdown(wait=True)                     # drenar y esperar

# Prioridades disponibles
Priority.CRITICAL   # 3
Priority.HIGH       # 2
Priority.NORMAL     # 1 (defecto)
Priority.LOW        # 0
```

FSM

```python
from rxnet.fsm import Machine, Transition, Runtime

# Transición mínima (sin guard = siempre dispara)
Transition(from_state=A, to_state=B)

# Transición completa
Transition(from_state=A, to_state=B, guard=my_guard, action=my_action)

# Máquina mínima
Machine(name="m", state=INITIAL, transitions=[...])

# Máquina con callbacks de fase
Machine(
    name="m",
    state=INITIAL,
    transitions=[...],
    user=my_data,
    latch_inputs_cb=read_inputs,    # fase latch: snapshot hardware
    dump_outputs_cb=write_outputs,  # fase dump: escribir hardware
)

rt = Runtime()
rt.add_machine(machine)                    # period_us=0: async (sin
   scheduling propio)
rt.add_machine(machine, period_us=10_000, deadline_us=10_000)
rt.tick()
```

Petri Net

```python
from rxnet.pn import Arc, Net, Transition, Runtime

# Arco
Arc(place_id=0, weight=1)    # consume/produce 1 token del lugar 0

# Transición
Transition(
    consume=[Arc(P_A, 1), Arc(P_B, 2)],  # necesita 1 de A y 2 de B
    produce=[Arc(P_C, 1)],               # produce 1 en C
    guard=my_guard,                       # opcional
    action=my_action,                     # deferred, opcional
)

# Red
Net(
    name="n",
    places=[1, 0, 0],           # marcado inicial
```

```
    transitions=[...],
    user=my_data,
    latch_inputs_cb=read_inputs,
    dump_outputs_cb=write_outputs,
)

rt = Runtime()
rt.add_net(net)                    # period_us=0: async (sin scheduling propio)
rt.add_net(net, period_us=10_000, deadline_us=10_000)
rt.tick()
```

Firmas de callbacks

```
from rxnet.runtime import Context

# Guard: pura, sin efectos, devuelve bool
def my_guard(ctx: Context, user: MyData) -> bool:
    return user.latched_event

# Acción: deferred, corre tras commit con estado ya comprometido
def my_action(ctx: Context, user: MyData) -> None:
    user.output = True

# Latch: snapshot hardware, actualizar señales derivadas
def latch_cb(ctx: Context, user: MyData) -> None:
    user.latched_event = read_gpio(user.button_pin)

# Dump: escribir salidas al hardware
def dump_cb(ctx: Context, user: MyData) -> None:
    write_gpio(user.led_pin, user.output)
```

---

9. Depuración y trazado

rxnet incluye un subsistema de trazado **totalmente opcional**: si no se activa, el código de producción es idéntico al que nunca supo que existía. No hay ramas extra, no hay comprobaciones de `None`, no hay coste en el camino caliente.

Cómo funciona

Al llamar a `Tracer.attach(rt)`, el tracer envuelve cada nodo con un proxy transparente (`_Traced`) que delega las cuatro fases al nodo original y, antes y después de cada una, escribe un evento de 16 bytes en un buffer circular pre-asignado. La estructura del runtime no se modifica: si el tracer nunca se adjunta, `Runtime`, `Machine` y `Net` no contienen ni una línea relacionada con el trazado.

`Tracer.attach(rt)` es además **idempotente e incremental**. Si se llama dos veces sobre el mismo runtime, no vuelve a envolver los nodos ya trazados. Si entre llamadas se añaden o eliminan nodos y se reconstruye el runtime, una nueva llamada a `attach()` conserva el identificador (`nid`) de los nodos ya conocidos y asigna identificadores nuevos solo a los nodos añadidos.

Uso básico

```
from rxnet import Tracer

tracer = Tracer(max_events=4096)
tracer.attach(rt)          # antes de ce.run() / te.run()

ce = CyclicExecutive()
ce.add(rt)
ce.run()                   # el sistema corre con trazado activo
```

Esto permite reconfigurar la topología durante la preparación del sistema sin recrear el tracer:

```
tracer.attach(rt)

# más tarde: cambia la red
rt.add_machine(aux_machine)
rt.build()

tracer.attach(rt)          # no duplica wrappers; solo adjunta lo nuevo
```

Mientras el sistema está en marcha, el tracer acumula eventos en el buffer circular. Cuando está lleno, los eventos más antiguos se descartan (se cuenta el número de eventos perdidos).

Descargar y visualizar la traza

Vía HTTP (sistema en ejecución)

```
tracer.serve(port=7777)    # daemon HTTP; llamar antes de ce.run()
```

Desde el Mac de desarrollo:

```
python -m rxnet.tools.trace http://target:7777 --report trace.html --
    open
```

Se genera un informe HTML independiente que se abre directamente en el navegador. Contiene:

- **Diagramas DOT** de cada FSM y red de Petri del sistema (renderizados en el navegador con Graphviz/WASM, sin instalación adicional).
- **Tabla WCRT** — tiempo de respuesta en caso peor por nodo.
- **Botón Perfetto** — abre la traza completa en ui.perfetto.dev como línea de tiempo interactiva donde se pueden inspeccionar activaciones, transiciones, duraciones de fase y eventos de usuario.

Desde un fichero

```
tracer.export("trace.bin")
```

```
python -m rxnet.tools.trace trace.bin --report trace.html --open
python -m rxnet.tools.trace trace.bin --stats          # WCRT por nodo
    en texto
python -m rxnet.tools.trace trace.bin --perfetto out.json
```

Trazado de fases (para docencia)

Con `phases=True` se registra el inicio y fin de cada fase individual (latch / evaluate / commit / dump), lo que permite medir y visualizar cuánto tarda cada una:

```
tracer = Tracer(max_events=8192, phases=True)
tracer.attach(rt)
```

Útil para identificar qué fase concentra el tiempo de respuesta o para explicar la ejecución del tick en un contexto educativo.

Eventos de usuario

Se pueden inyectar eventos arbitrarios desde cualquier hilo:

```
tracer.user("temperatura", 42)
tracer.user("alarma", 1)
```

Aparecen en la línea de tiempo de Perfetto como eventos instantáneos globales, alineados con las activaciones de los nodos.

Nombres de estados y lugares

Para que los diagramas y la traza muestren nombres legibles en lugar de índices numéricos, declara `state_names` en `Machine` y `place_names` / `transition_names` en `Net`:

```
Machine(
    name="semaforo",
    state=ROJO,
    state_names={ROJO: "ROJO", VERDE: "VERDE", AMBAR: "ÁMBAR"},
    transitions=[...],
)

Net(
    name="buffer",
    places=[1, 0],
    place_names={0: "LIBRE", 1: "OCUPADO"},
    transition_names=["adquirir", "liberar"],
    transitions=[...],
)
```

Retirar el trazado

```
tracer.detach(rt)   # restaura los nodos originales; overhead = cero de nuevo
```

Si después de `detach()` se modifica la estructura del runtime, se puede volver a llamar a `attach()` y el tracer reutilizará los `nid` de los nodos ya conocidos, asignando identificadores nuevos únicamente a los nodos que no hubieran sido vistos antes.

---

10. Extracción de diagramas y verificación formal

La herramienta `rxnet-extract` lee directamente el código fuente Python y genera diagramas de estado o una especificación TLA+ sin ningún paso intermedio. No hace falta mantener YAML ni ficheros de anotaciones adicionales.

Instalación

```
pip install rxnet          # instala rxnet-extract en $PATH
# o bien:
brew install rxnet-python
```

No requiere dependencias externas para ficheros `.py` — usa el módulo `ast` de la stdlib de Python.

Uso desde la línea de comandos

```
# Graphviz DOT → SVG / PDF
rxnet-extract light_fsm.py --dot | dot -Tsvg -o light.svg
rxnet-extract light_fsm.py --dot | dot -Tpdf -o light.pdf

# Mermaid stateDiagram-v2 (se renderiza en GitHub Markdown)
rxnet-extract light_fsm.py --mermaid

# TLA+ para model checking con TLC
rxnet-extract light_fsm.py --tla > light_fsm.tla

# Varios formatos en un solo paso (separados por '---')
rxnet-extract light_fsm.py --tla --dot --mermaid
```

La misma herramienta acepta ficheros `.c` si `clang` está en el `PATH`:

```
rxnet-extract light_fsm.c -Iinclude --dot | dot -Tsvg -o light.svg
```

Cómo funciona

`rxnet-extract` usa `ast.parse()` de la stdlib y recorre el AST en busca de tres patrones:

1. Asignaciones `*_STATE_* = int` en el módulo → tabla de estados.
2. Lista `[Transition(from, to, guard=fn, action=fn), ...]` → transiciones.
3. Llamada `Machine(name="...", state=CONST, ...)` → nombre de la máquina y estado inicial.

Los guiones bajos iniciales de los nombres de función se eliminan automáticamente: `_button_pressed` aparece como `button_pressed` en la salida.

Ejemplo de salida Mermaid

```
stateDiagram-v2
    [*] --> OFF
    OFF --> ON : button_pressed / light_on
    ON --> OFF : button_pressed / light_off
```

Restricción: solo patrones estáticos

La extracción es estática: `rxnet-extract` no ejecuta el código. Si los estados o transiciones se construyen dinámicamente (listas generadas en runtime, clases que heredan estados, etc.) la herramienta no los verá. El patrón estándar rxnet — constantes de módulo + lista literal de `Transition` + llamada `Machine` — siempre es extraíble.

Lectura adicional

- **Código fuente**: `python/rxnet/` — runtime (~80 líneas), fsm (~80 líneas), pn (~130 líneas)
- **Tests**: `python/tests/` — 77 tests que documentan el comportamiento esperado
- **Ejemplos interactivos**: `python/examples/fsm/` y `python/examples/pn/`
- **Especificaciones**: `docs/specs/python/requirements.md` y `docs/specs/python/design.md`
- **Implementación C**: `c/` — misma semántica, API equivalente en C11